Activities Google Chrome

Mail - Ritik Patidar - Ou | HackerRank

hackerrank.com/test/gecgbb915mk/questions/dff46tghhk0

Apps | Interview Pre... | machine learni... | CS6910/CS70... | Computer Scie...

58m left

ALL

1

2

1. Count Swaps During Custom Sorting

The following algorithm is proposed to be used to sort an array of distinct *n* integers:

For the input array *arr* of size *n* do:

- Try to find the smallest pair of indices $0 \le i < j \le n-1$ such that *arr[i] > arr[j]*. Here smallest means usual alphabetical ordering of pairs, i.e. $(i_1, j_1) < (i_2, j_2)$ if and only if $i_1 < i_2$ or ($i_1 = i_2$ and $j_1 < j_2$).
- If there is no such pair, stop.
- Otherwise, swap *a[i]* and *a[j]* and repeat finding the next pair.

The algorithm seems to be correct, but the question is how efficient is it? Write a function that returns the number of swaps performed by the above algorithm.

For example, if the initial array is *[5,1,4,2]*, then the algorithm first picks pair *(5,1)* and swaps it to produce array *[1,5,4,2]*. Next, it picks pair *(5,4)* and swaps it to produce array *[1,4,5,2]*. Next, pair *(4,2)* is picked and swapped to produce array *[1,2,5,4]*, and finally, pair *(5,4)* is swapped to produce the final sorted array *[1,2,4,5]*, so the number of swaps performed is *4*.

Language Python 3

```
1  > #!/bin/py
10
11    #
12    # Complete
13    #
14    # The func
15    # The func
16    #
17
18    def howMan
19        # Writ
20
21  > if __name__
```

Test Results

produce array *[1,5,4,2]*. Next, it picks pair *(5,4)* and swaps it to produce array *[1,4,5,2]*. Next, pair *(4,2)* is picked and swapped to produce array *[1,2,5,4]*, and finally, pair *(5,4)* is swapped to produce the final sorted array *[1,2,4,5]*, so the number of swaps performed is *4*.

Function Description

Complete the function *howManySwaps* in the editor below. The function should return an integer that denotes the number of swaps performed by the proposed algorithm on the input array.

The function has the following parameter(s):

arr: integer array of size n with all unique elements

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq arr[i] \leq 10^9$
- *all elements of arr are unique*

▼ Input Format Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

In the first line, there is a single integer *n*.
In the *i-th* of the next *n* lines, there is a single integer *arr[i]*.

▼ Sample Case 0

Language Pytho

```
1  > #!/bin/
10
11   #
12   # Comple
13   #
14   # The fu
15   # The fu
16   #
17
18   def howMa
19       # Wri
20
21 > if __name
```

Test Results

Activities

Mail - Ritik Patidar - Outl

HackerRank

hackerrank.com/test/gecgbb915mk/questions/dff46tghhk0

Apps | Interview Pre... | machine learni... | CS6910/CS70... | Computer Scie...

58m left

integer *arr[i]*.

Sample Case 0

Sample Input

```
3
7
1
2
```

Sample Output

```
2
```

Explanation

There are 3 elements in the array, 7, 1 and 2 respectively.

- Initially, there are two pairs of indices $i < j$ for which $a[i] > a[j]$. These pairs are (0, 1) and (0, 2). Since (0, 1) is smaller of them, the algorithm swaps elements *a[0]* and *a[1]*. The resulting array is [1, 7, 2].
- Next, in the second iteration there is only a single pair of indices $i < j$ for which $a[i] > a[j]$. This pair is (1, 2) and the algorithm swaps *a[1]* with *a[2]*. The resulting array is [1, 2, 7].
- After that, the algorithm tries to find the next pair of indices to swap but since there is none, the algorithm stops.

The number of swaps it performed is 2.

Sample Case 1

Sample Input

Language Python 3

```
1  > #!/bin/pyth
10
11    #
12    # Complete
13    #
14    # The funct
15    # The funct
16    #
17
18    def howManyS
19        # Write
20
21  > if __name__
```

Test Results

Activities Google Chrome

Mail - Ritik Patidar - Outl | HackerRank

hackerrank.com/test/gecgbb915mk/questions/dff46tghhk0

Apps | Interview Pre... | machine learni... | CS6910/CS70... | Computer Scie...

58m left

ALL

1

2

There are 3 elements in the array, 7, 1 and 2 respectively.

- Initially, there are two pairs of indices $i < j$ for which $a[i] > a[j]$. These pairs are (0, 1) and (0, 2). Since (0, 1) is smaller of them, the algorithm swaps elements *a[0]* and *a[1]*. The resulting array is [1, 7, 2].
- Next, in the second iteration there is only a single pair of indices $i < j$ for which $a[i] > a[j]$. This pair is (1, 2) and the algorithm swaps *a[1]* with *a[2]*. The resulting array is [1, 2, 7].
- After that, the algorithm tries to find the next pair of indices to swap but since there is none, the algorithm stops.

The number of swaps it performed is 2.

▼ Sample Case 1

Sample Input

```
2
7
12
```

Sample Output

```
0
```

Explanation

There are 2 elements in the array, 7 and 12. The algorithm cannot find any pair in its main loop, so it does not perform any swaps and stops. The number of performed swaps is 0.

Language Python 3

```
1  > #!/bin/pytho
10
11   #
12   # Complete t
13   #
14   # The functio
15   # The functio
16   #
17
18   def howManySw
19       # Write yo
20
21 > if __name__ ==
```

Test Results Cust

57m left

ALL

1

2

2. Caesar Edit Distance

Given two strings *source* and *target*, both of length *n* and consisting of lowercase English letters, find the minimum *edit distance* between the two strings. Edit distance refers to the minimum number of insert and delete operations required to transform one string into the other. For example, the minimum edit distance between the strings *aba* and *caa* is *2:*

- *aba:* delete *b* to get *aa*
- *aa:* insert *c* to get *caa*, the target value.

There are two steps:

- Encipher *source* using the *Caesar cipher* to produce *source'*. Choose any non-negative integer as a *key*. In the Caesar cipher, the alphabet is 'shifted' by that number of places. The alphabet is treated as a circular list, so in a shift of +1, *z* is replaced by the letter *a*.

+0	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
+1	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a
+2	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	a	b

- Determine the edit distance between *source* and *target*.

Example

Language Python

```
1  > #!/bin/py
10
11   #
12   # Complete
13   #
14   # The func
15   # The func
16   #  1. STRI
17   #  2. STRI
18   #
19
20   def editDis
21       # Write
22
23 > if __name__
```

Test Results

Activities

Mail - Ritik Patidar - Outl

HackerRank

hackerrank.com/test/gecgbb915mk/questions/2mbc0blbpqk

Apps | Interview Pre... | machine learni... | CS6910/CS70... | Computer Scie...

57m left

ALL

1

2

Example

source = "abdh"

target = "bcif"

Shift *source = "abdh"* by *1* to get *source' = "bcei"*. Calculate the edit distance between *source'* and *target.*

- *"bcei":* remove the letter *e* to get *source' = "bci"*
- *"bci":* add the character *f* to have *"bcif"*

It took one deletion and one insertion, so the edit distance is 2. There is no other shift that allows a lower edit distance in this case.

Function Description

Complete the function *editDistance* in the editor below.

editDistance has the following parameter(s):

string source: the first string

string target: the second string

Returns:

int: an integer that represents the the minimal edit distance achievable.

Constraints

- $1 \leq n \leq 10^3$
- The strings *source* and *target* both contain only characters in the range ascii[a-z].

Language Python 3

```
1  > #!/bin/pyt
10
11   #
12   # Complete
13   #
14   # The funct
15   # The funct
16   #   1. STRIN
17   #   2. STRIN
18   #
19
20   def editDist
21       # Write
22
23 > if __name__
```

Test Results

57m left

ALL

1

2

Input Format For Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains a string, *source*.
The next line contains a string, *target*.

Sample Case 0

Sample Input 0

```
STDIN   Function
-----   --------
ab   →  source = "ab"
ef   →  target = "ef"
```

Sample Output 0

```
0
```

Explanation 0

Shift *source* by *4* to get *source'* = *"ef"*. Since this matches the target string, the edit distance is 0.

Sample Case 1

Sample Input 1

```
STDIN    Function
-----    --------
abc   →  source = "abc"
gzu   →  target = "gzu"
```

Sample Output 1

57m left

ALL

1

2

▼ Sample Case 1

Sample Input 1

```
STDIN       Function
-----       --------
abc     →   source = "abc"
gzu     →   target = "gzu"
```

Sample Output 1

```
4
```

Explanation 1

The best one can do is to shift *source* until one character matches in *target*, then remove and insert 2 characters. The edit distance is 4.

▼ Sample Case 2

Sample Input 2

```
STDIN Function
----- --------
www   →  source = "www"
ssh →  target = "ssh"
```

Sample Output 2

```
2
```

Explanation 2

Shift *source* by 22 to have *source'* = "sss".
Remove and insert 1 character. The edit distance is 2.